

# Local GCV, Synchronized Local Model Selection, and LPS

Source path: ~/current\_projects/geosmooth/split\_handoffs/  
lps\_gcv\_synchronized\_selection.2026-06-05/  
lps\_gcv\_synchronized\_selection\_design.tex

Build: 2026-06-05 12:43:24 EDT

## Purpose

This report records a systematic version of the GCV idea for local polynomial smoothing (LPS). The motivating question is:

Can local weighted-linear-model diagnostics guide the choice of neighborhood size, kernel, degree, and chart dimension?

The immediate application is the comparison between

```
chart.dim = "auto"    and    chart.dim = "local.auto".
```

The broader methodological idea is more general. Every local chart fit in LPS is a weighted linear regression problem. Therefore each local fit has linear-model diagnostics such as residual sum of squares, leverage, effective degrees of freedom, and generalized cross-validation (GCV). These quantities may help choose local modeling parameters in a data-driven way.

The key conceptual turn is this: local GCV should not only be viewed as an isolated diagnostic for one chart. It can also be aggregated, smoothed, or synchronized across overlapping neighborhoods. This creates a bridge between ordinary LPS, local parameter selection, and the synchronization philosophy that motivated S-LPL-TF / SLPLiT.

## 1 The Local Weighted Polynomial Fit

Let  $x_1, \dots, x_n$  be observed covariate points and let

$$y_i = f_i + \varepsilon_i$$

be the observed response at point  $i$ . For an anchor point  $i$ , LPS chooses a local support

$$N_i \subset \{1, \dots, n\}, \quad m_i = |N_i|.$$

The local support is usually determined by a neighborhood size, a metric, and a kernel bandwidth rule. After constructing a local chart around the anchor, each neighbor  $j \in N_i$  has local coordinates

$$z_{ij} \in \mathbb{R}^{d_i},$$

where  $d_i$  is the chart dimension used at anchor  $i$ . For polynomial degree  $p_i$ , form the local polynomial design matrix

$$X_i \in \mathbb{R}^{m_i \times q_i},$$

where each row contains the polynomial basis evaluated at  $z_{ij}$ . For example, with degree one,

$$\phi(z) = (1, z_1, \dots, z_{d_i}),$$

so  $q_i = d_i + 1$ . With degree two, the design also includes all squared and cross-product terms.

Let

$$W_i = \text{diag}(w_{ij} : j \in N_i)$$

be the diagonal matrix of nonnegative kernel weights, and let

$$y_{N_i} = (y_j : j \in N_i)$$

be the local response vector. The local weighted least-squares estimate is

$$\hat{\beta}_i = \arg \min_{\beta \in \mathbb{R}^{q_i}} \|W_i^{1/2}(y_{N_i} - X_i\beta)\|_2^2.$$

When the weighted design has full rank, this is

$$\hat{\beta}_i = (X_i^\top W_i X_i)^{-1} X_i^\top W_i y_{N_i}.$$

In implementation, this should be computed by a stable QR or SVD-based solve rather than by forming an explicit inverse.

## 2 Local Hat Matrix, Degrees of Freedom, and GCV

For fixed local choices, the fitted values on the local support are a linear function of the local response vector:

$$\hat{y}_{N_i} = H_i y_{N_i},$$

where

$$H_i = X_i(X_i^\top W_i X_i)^{-1} X_i^\top W_i.$$

The matrix  $H_i$  is the local weighted hat matrix. It describes how the observed local responses are transformed into local fitted values. Its trace is the effective local degrees of freedom:

$$\text{df}_i = \text{tr}(H_i).$$

For ordinary unweighted least squares with a full-rank design,  $\text{df}_i$  is the number of fitted coefficients. With weights, numerical rank deficiency, or regularization,  $\text{df}_i$  is the effective model complexity.

The local residual sum of squares is

$$\text{RSS}_i = \|W_i^{1/2}(y_{N_i} - \hat{y}_{N_i})\|_2^2.$$

A local generalized cross-validation score can then be written as

$$\text{GCV}_i = \frac{\text{RSS}_i / m_i}{(1 - \text{df}_i / m_i)^2}.$$

This score penalizes models that fit the local data well only by using too much local flexibility. If  $\text{df}_i / m_i$  is close to one, the denominator becomes small and the score becomes large. That is exactly the desired behavior when a local fit is close to being underdetermined or interpolating.

### 3 The Caveat and the Opportunity

The caveat is that  $\text{GCV}_i$  estimates prediction behavior inside the local neighborhood  $N_i$ . It is not automatically the same thing as the global held-out prediction error used by dataset-level cross-validation. Neighboring local supports overlap heavily, so the local GCV scores

$$\text{GCV}_1, \dots, \text{GCV}_n$$

are dependent rather than independent observations.

This caveat is also an opportunity. The dependence is not merely a nuisance; it is information about the geometry of the smoother. If nearby anchors share many data points and produce related local models, then their GCV scores and their selected local parameters should not be treated as unrelated. Instead, we can use overlap and graph adjacency to stabilize local model selection.

This is closely aligned with the motivation behind S-LPL-TF / SLPLiFT. In S-LPL-TF, the synchronization term was introduced because neighboring local models should agree when they make predictions on overlapping regions. The GCV view gives a complementary way to express a similar desire: local models should not only fit well locally, but the field of local choices should be stable and coherent across the data geometry.

### 4 Formulation 1: Global Discrete GCV Selector

The simplest formulation keeps all tuning parameters global. Let

$$\theta = (k, \kappa, p, d)$$

denote a candidate parameter setting, where:

- $k$  is the neighborhood/support size;
- $\kappa$  is the kernel type, such as Gaussian or tricube;
- $p$  is the polynomial degree;
- $d$  is the local chart dimension.

For a fixed global  $\theta$ , every anchor uses the same rule. Each anchor still has its own local support, design matrix, and weights, but the model configuration is global. Compute

$$\text{GCV}_i(\theta), \quad i = 1, \dots, n.$$

Then select

$$\hat{\theta}_{\text{global}} = \arg \min_{\theta \in \Theta} \sum_{i=1}^n \text{GCV}_i(\theta).$$

This is the most auditable first experiment. It is a direct analogue of cross-validation over a candidate grid, but it replaces fold refits with local linear-model diagnostics. It should be much cheaper than full K-fold CV and easier to inspect pointwise.

**Robust aggregation.** Instead of summing raw GCV scores, one may use a robust transformation:

$$\hat{\theta}_{\text{robust}} = \arg \min_{\theta \in \Theta} \sum_{i=1}^n \rho\{\text{GCV}_i(\theta)\}.$$

Here  $\rho$  is increasing but reduces the influence of extreme local failures. Examples include a winsorized score, a Huberized score, or a trimmed mean. This may matter on real geometries where a few local neighborhoods are numerically unstable.

## 5 Formulation 2: Local GCV with a Graph-Smoothed Parameter Field

The next formulation lets tuning parameters vary by anchor:

$$\theta_i = (k_i, \kappa_i, p_i, d_i).$$

If each anchor independently minimizes its own GCV,

$$\hat{\theta}_i^{\text{greedy}} = \arg \min_{\theta_i \in \Theta_i} \text{GCV}_i(\theta_i),$$

then the resulting parameter field may be noisy. Nearby anchors could choose very different dimensions or supports because of small local fluctuations.

To borrow strength across nearby anchors, put a graph on the data points:

$$G = (V, E), \quad V = \{1, \dots, n\}.$$

Then choose the full parameter field

$$\theta_{1:n} = (\theta_1, \dots, \theta_n)$$

by minimizing

$$\sum_{i=1}^n \text{GCV}_i(\theta_i) + \eta \sum_{(i,j) \in E} D(\theta_i, \theta_j).$$

Here  $\eta \geq 0$  controls how strongly neighboring local choices are synchronized, and  $D$  measures disagreement between local parameter choices.

For example, if the most important local choice is chart dimension, one could use

$$D(\theta_i, \theta_j) = (d_i - d_j)^2.$$

If support size matters too, one might use

$$D(\theta_i, \theta_j) = \alpha_d (d_i - d_j)^2 + \alpha_k \{\log(k_i) - \log(k_j)\}^2 + \alpha_\kappa \mathbf{1}\{\kappa_i \neq \kappa_j\}.$$

The exact choice of  $D$  is a modeling decision. The important point is that this formulation selects a spatially coherent field of local model choices, rather than a collection of independent local winners.

## 6 Formulation 3: Prediction-Synchronized LPS

The most S-LPL-TF-like formulation synchronizes predictions rather than only the discrete parameter choices. Let

$$\widehat{f}^{(i)}(x)$$

denote the local polynomial model fitted at anchor  $i$ , evaluated at a point  $x$  where that local model is meaningful. If anchors  $i$  and  $j$  have overlapping supports, then both local models can often predict values on the same points. A prediction-synchronization penalty can be written schematically as

$$\sum_{(i,j) \in \mathcal{O}} \sum_{x_\ell \in N_i \cap N_j} \omega_{ij\ell} \left\{ \widehat{f}^{(i)}(x_\ell) - \widehat{f}^{(j)}(x_\ell) \right\}^2,$$

where  $\mathcal{O}$  is a set of overlapping anchor pairs and  $\omega_{ij\ell}$  are overlap weights.

A prediction-synchronized LPS criterion could therefore be

$$\sum_{i=1}^n \text{GCV}_i(\theta_i) + \eta \sum_{(i,j) \in \mathcal{O}} \sum_{x_\ell \in N_i \cap N_j} \omega_{ij\ell} \left\{ \widehat{f}_{\theta_i}^{(i)}(x_\ell) - \widehat{f}_{\theta_j}^{(j)}(x_\ell) \right\}^2.$$

This has the same spirit as S-LPL-TF but removes the  $\ell_1$  lifting trend filtering penalty. In words: choose local models that have good local predictive diagnostics and also agree with nearby local models on overlapping regions.

## 7 Relation to S-LPL-TF / SLPLiT

S-LPL-TF introduced synchronization because local polynomial lifting models should not behave as unrelated fits. Each local model gives predictions on a support. When supports overlap, disagreement between those predictions is evidence that the local models are not forming a coherent global smoother.

The GCV idea complements this. It says:

First, ask whether each local model is statistically reasonable.

Then synchronization says:

Second, ask whether neighboring local models are mutually coherent.

So there are two kinds of regularity:

1. **Internal local regularity:** a local model should not use too much flexibility for its neighborhood. GCV, degrees of freedom, leverage, and condition diagnostics address this.
2. **External overlap regularity:** neighboring local models should not make incompatible predictions on shared data. Synchronization penalties address this.

This suggests a larger design principle: local smoothers should balance local fit, local complexity, and overlap coherence.

## 8 Relation to Batched or Randomized Optimization

The analogy with minibatch optimization in neural networks is useful, but it should be stated carefully. The similarity is not that LPS-GCV selection requires gradient descent. The similarity is that a large global objective is assembled from many local losses:

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell_i(\theta).$$

In neural-network training, the local losses are usually sample losses, and minibatches provide noisy but computationally manageable estimates of the full objective. In LPS-GCV selection, the local losses are neighborhood-level GCV scores. One could imagine estimating

$$\sum_{i=1}^n \text{GCV}_i(\theta)$$

using random batches of anchor points rather than all anchors at every candidate evaluation.

This gives a possible computational strategy:

1. sample a batch of anchors  $B \subset \{1, \dots, n\}$ ;
2. evaluate  $\sum_{i \in B} \text{GCV}_i(\theta)$  for candidate  $\theta$ ;
3. refine or eliminate candidate parameters based on repeated batches;
4. only run full-anchor evaluation for the most promising candidates.

The statistical motivation is also similar: do not trust one highly local quantity too much. Use many local pieces of evidence, possibly batched, smoothed, or synchronized, to make more stable choices.

## 9 Recommended First Test

Before building the more ambitious synchronized versions, the first test should be simple and auditable.

### Test 1: Global discrete GCV selector on frozen local-auto batch

For each frozen dataset and each candidate

$$\theta = (k, \kappa, p, d),$$

compute

$$S_{\text{GCV}}(\theta) = \sum_{i=1}^n \text{GCV}_i(\theta).$$

Then compare the selected candidate

$$\hat{\theta}_{\text{GCV}} = \arg \min_{\theta} S_{\text{GCV}}(\theta)$$

to the candidates selected by the existing CV-based auto and local-auto runs.

The output should include:

- the GCV-selected support size, kernel, degree, and chart dimension;
- the CV-selected support size, kernel, degree, and chart dimension;
- Truth RMSE for both selected fits when truth is available;
- local degrees-of-freedom ratios  $\text{df}_i / m_i$ ;
- local condition diagnostics;
- a comparison between pointwise Truth-RMSE contributions  $c_i$  and local GCV diagnostics.

The key question is whether red  $c_i$  regions correspond to high GCV, high leverage, high  $\text{df}_i / m_i$ , or unstable local designs. If the answer is yes, local GCV is not only a selector candidate; it is also an explanatory diagnostic for why one chart-dimension rule wins or loses.

## 10 Possible Next Tests After the First One

If the global GCV selector is promising, the next natural steps are:

1. replace the raw sum of GCV scores by robust summaries, such as trimmed means or Huberized sums;
2. allow chart dimension to vary by anchor while keeping support size and kernel global;
3. add graph-smoothed penalties on the local dimension field;
4. compare prediction-synchronized LPS to S-LPL-TF without the  $\ell_1$  lifting term;
5. test batched anchor subsets as a scalable approximation for large 16S-style datasets.

## Conclusion

Local GCV provides a principled way to use the fact that each local LPS chart is a weighted linear model. The simplest version selects one global parameter setting by minimizing the sum of local GCV scores. The more interesting versions treat local GCV scores as a field over the data geometry and stabilize that field through graph smoothing or prediction synchronization. This is conceptually close to the synchronization motivation behind S-LPL-TF, but it focuses first on local statistical adequacy rather than on trend-filtering structure.

The recommended first implementation is therefore deliberately modest: compute local GCV diagnostics on the frozen non-manifold batch and test whether a global GCV selector agrees with, improves on, or explains the existing CV-selected auto and local-auto fits.

## A Weighted Hat Matrix Details

The weighted least-squares problem at anchor  $i$  is

$$\min_{\beta} (y_{N_i} - X_i \beta)^\top W_i (y_{N_i} - X_i \beta).$$

Assuming  $X_i^\top W_i X_i$  is nonsingular, the solution is

$$\hat{\beta}_i = (X_i^\top W_i X_i)^{-1} X_i^\top W_i y_{N_i}.$$

The local fitted vector is

$$\hat{y}_{N_i} = X_i \hat{\beta}_i = X_i (X_i^\top W_i X_i)^{-1} X_i^\top W_i y_{N_i}.$$

Thus

$$H_i = X_i (X_i^\top W_i X_i)^{-1} X_i^\top W_i.$$

The local degrees of freedom are

$$\text{df}_i = \text{tr}(H_i).$$

For numerical implementation, it is often better to work with

$$\tilde{X}_i = W_i^{1/2} X_i, \quad \tilde{y}_i = W_i^{1/2} y_{N_i}.$$

Then the fitted values in the weighted space are obtained by projecting  $\tilde{y}_i$  onto the column space of  $\tilde{X}_i$ . If  $\tilde{X}_i$  has singular value decomposition

$$\tilde{X}_i = U_i D_i V_i^\top,$$

then the effective rank, leverage diagnostics, and degrees of freedom can be computed stably from this decomposition. This is also where rank tolerance and conditioning diagnostics enter naturally.

## B Why Ordinary GCV Can Fail Locally

Local GCV can fail or become unstable when:

- $m_i$  is too small relative to the number of local polynomial coefficients;
- the local design is nearly rank deficient;
- kernel weights concentrate on too few effective observations;
- the local neighborhood crosses a singularity, boundary, or mixed dimension transition;
- the local chart dimension is overestimated.

These are not reasons to discard GCV. They are reasons to record diagnostics alongside it. In fact, if high GCV or high  $\text{df}_i / m_i$  aligns with red pointwise Truth-RMSE contributions, then GCV is doing exactly what we want: it is identifying local modeling choices that are too flexible or unstable.